JobMate AI: Personal Placement Strategist

Comprehensive System & Technical Architecture Report

Executive Summary

JobMate AI is an intelligent, offline-capable placement preparation platform designed to bridge the gap between candidate resumes and specific job requirements. Utilizing local Large Language Model (LLM) inference through **Ollama (`qwen2.5:3b`)**, PyMuPDF document extraction, regex-based skill detection, and persistent SQLite tracking, the application provides candidates with actionable skill-gap diagnostics, custom day-by-day learning schedules, and gamified multi-level technical assessments.

Originally developed as a monolithic single-script prototype, the system has been refactored into a **modular, production-ready multi-tier architecture** ensuring maintainability, separation of concerns, and cross-platform compatibility on modern Python runtimes (including Python 3.13).

1. Problem Statement & Objectives

1.1 The Challenge

Job seekers frequently apply to technical roles without a clear understanding of:

1. How closely their resumes match specific job descriptions (JDs).
2. What critical skill gaps must be closed before technical interviews.
3. How to structure their preparation within constrained timeframes (e.g., 7 to 30 days).
4. Their actual proficiency in foundational database and role-specific competencies.

1.2 System Objectives

- **Automated Gap Analysis**: Ingest PDF resumes, extract selectable text, and detect overlap/discrepancies against target job postings.
- **Deterministic & Generative Recommendations**: Combine keyword heuristics with local LLM reasoning to create realistic, day-numbered study roadmaps.
- **Progressive Technical Assessments**: Offer structured SQL quiz levels with sequential unlocking based on an 80% passing threshold, alongside targeted quizzes for specific roles (Data Analyst, Software Engineer, Data Scientist, Web Developer).

- **Privacy & Local Execution**: Operate fully offline without sending user resumes or proprietary job postings to third-party cloud APIs.
- **Persistent Progress Tracking**: Record test attempts, historical scores, timestamps, and feedback in a local SQLite database.

2. System Architecture

The project follows a modular, decoupled architecture where presentation, business logic, data persistence, and external service layers are strictly separated.

3. Modular File Structure

File	Primary Responsibility	Key Functions / Entities
	Application entry point, layout controller, and view router.	Tabbed vs. Single-page mode toggling, session state initialization.
	Application-wide constants, model configurations, and skill lists.	`OLLAMA_URL`, `MODEL`, `DB_PATH`, `SKILLS`, `PASS_PERCENTAGE`.
	Static assessment repositories for SQL levels and role quizzes.	`QUIZ_LEVELS` (Basics, Aggregation, Joins), `ROLE_QUIZZES`.
	SQLite persistence, scoring mechanisms, and unlock algorithms.	`initialize_database()`, `save_quiz_result()`, `get_unlocked_levels()`, `get_recommendation()`.
	I/O parsing, text processing, prompt engineering, and LLM calls.	`extract_resume_text()`, `detect_skills()`, `call_ollama()`, `build_plan_prompt()`.
	Reusable Streamlit interface blocks and interactive forms.	`render_placement_analyzer()`, `render_sql_missions()`, `render_role_missions()`, `render_learning_progress()`.
	Python dependencies pinned for runtime stability.	`numpy>=2.1.0`, `streamlit>=1.40.0`, `pymupdf>=1.24.0`, `requests>=2.31.0`.

4. Core Functional Modules

4.1 Resume Ingestion & Skill Detection

- **Text Extraction**: Uses PyMuPDF (`fitz`) to read binary byte streams directly from Streamlit memory without writing intermediate files to disk.
- **Skill Matching**: Utilizes word-boundary regular expressions (`rf"(?<!\w){re.escape(skill)}(?!\w)"`) against a curated catalog of technical competencies. This prevents false substring positives (e.g., matching `"c"` inside `"excel"`).
- **Match Scoring**: Calculates the percentage of required JD skills detected on the candidate's resume:

$$\text{Match Percentage} = \text{round}(\frac{|\text{Matched Skills}|}{|\text{Required Skills}|} \times 100)$$

Match Percentage = round (|Required Skills||Matched Skills| × 100)

4.2 Generative Study Plan Generation

- Constructs a strict prompt containing detected resume skills, confirmed matches, identified skill gaps, candidate resume text (up to 8,000 characters), and job description text (up to 5,000 characters).
- Prompts the local `qwen2.5:3b` model to create a day-by-day plan covering exactly the target preparation window, enforcing practical exercises and verification checks for every day.

4.3 Gamified SQL & Role-Based Assessments

- **Sequential SQL Mastery**: Evaluates user responses to 5-question problem sets across three distinct difficulty tiers:
    1. *SQL Basics* (SELECT, WHERE, ORDER BY, COUNT, DISTINCT)
    2. *SQL Aggregation* (GROUP BY, HAVING, SUM, AVG)
    3. *SQL Joins & Subqueries* (INNER/LEFT JOIN, ON clauses, subqueries)
- **Progression Locking**: Enforces an 80% passing threshold (`PASS_PERCENTAGE = 0.80`) on prerequisite tiers before unlocking subsequent challenges.
- **Target-Role Missions**: Includes customized assessments mapped to industry roles (Data Analyst, Software Engineer, Data Scientist, Web Developer). The candidate's target role entered in the placement section dynamically presets the relevant assessment.

4.4 Persistent Local Database

- SQLite table `quiz_results` stores historical quiz data:
    - `id` (Primary Key Autoincrement)
    - `quiz_name` (Text)

    - `score` (Integer)
    - `total` (Integer)
    - `recommendation` (Text)
    - `created_at` (Timestamp Default `CURRENT_TIMESTAMP`)
- Tracks score trends (improved, declined, or maintained) across attempts and visualizes the last 10 attempts via interactive tables.

5. Technical Challenges & Engineering Solutions

```
```

Challenge 1: Monolithic State & Re-render Overlaps

- **Problem**: In the original script, all forms, database queries, and AI calls were in one continuous file. State updates and form resubmissions forced full-page layout jumps.
- **Solution**: Separated forms into modular components in `ui_components.py` with individual form keys and created a sidebar layout switcher enabling users to select between a modern **Tabbed View** and the original **Single Page View**.

Challenge 2: Python 3.13 C-Extension Build Failures (NumPy 1.x)

- **Problem**: Attempting to install older Streamlit dependencies triggered an uncompiled source build (`numpy-1.26.4.tar.gz`) requiring Cython and Microsoft Visual C++ Build Tools, which failed on Windows with Python 3.13.
- **Solution**: Updated `requirements.txt` to require `numpy>=2.1.0` and `streamlit>=1.40.0`, which supply pre-compiled binary wheels (`.whl`) specifically compiled for CPython 3.13 on Windows.

Challenge 3: Windows PATH Configuration

- **Problem**: Python was installed in user application directories without system PATH registration, leading to `pip : The term 'pip' is not recognized` errors.
- **Solution**: Provided both direct invocation syntax (`& "path\python.exe" -m pip ...`) and a PowerShell environment patch script to register Python and its `Scripts` directory into the user `PATH`.

6. Technology Stack Summary

Layer	Technology	Reason for Selection
Language	Python 3.13	Modern syntax, robust typing, high performance.
Frontend / Web UI	Streamlit (>= 1.40.0)	Rapid reactive dashboarding, native session state, zero frontend boilerplate.
PDF Extraction	PyMuPDF / fitz (>= 1.24.0)	Fast C-backed PDF parsing with direct byte stream support.
Local LLM Engine	Ollama (`qwen2.5:3b`)	Lightweight, high reasoning density, 100% offline data privacy.
Database	SQLite 3	Zero-configuration, zero-dependency relational local persistence.
Networking	Requests (>= 2.31.0)	Reliable HTTP communication with Ollama's local REST API.

7. Future Roadmap & Recommendations

1. **Semantic Embeddings for Skill Extraction**:
    - *Current*: Regex dictionary lookup against 24 common keywords.
    - *Proposed*: Use sentence-transformers or embedding models (e.g., `all-MiniLM-L6-v2` or `nomic-embed-text`) with cosine similarity to capture unlisted synonyms and soft skills.
2. **Dynamic AI Quiz Generation**:
    - Allow Ollama to generate dynamic technical questions directly based on the candidate's exact identified skill gaps rather than using static question banks.
3. **ATS Formatting Reviewer**:
    - Add automated checks for resume readability, typography, standard section headers, and ATS parseability ratings.
4. **Exportable Study Roadmaps**:
    - Enable one-click export of the AI-generated day-by-day plan to Markdown, PDF, or Google Calendar / iCal events.